

Bridging Contract Intelligence and Approval Governance: A Database-Enforced Approach to Sequential Workflow and Tamper-Evident Auditing

Ms. Ramala T
Assistant Professor
Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
ramala.t@rajalakshmi.edu.in

Veronica Regina Paul
Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701377@rajalakshmi.edu.in

Tharshini M
Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701361@rajalakshmi.edu.in

Swathi Saravanan
Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701356@rajalakshmi.edu.in

Vetriselvi M
Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701379@rajalakshmi.edu.in

ABSTRACT

Enterprise contract management fails at its structural seam: document intelligence and approval governance are separate domains, so risks are detected but never reach the stakeholders who must act on them. Existing remedies either impose the cost of a distributed ledger or rely on loose coupling at the application level that depends on human discipline. An architecture for single-enterprise contract lifecycle management (CLM) is proposed that couples a database-enforced sequential approval state machine with vector-based document intelligence. An audit-trail of tamper-evident events is achieved without the overhead of distributed consensus, using a SHA-256 hash-chained audit log in PostgreSQL under a trusted-administrator trust model. Clause classification uses a hybrid pipeline combining rule-based heuristics with cosine similarity against 40+ CUAD-inspired clause prototypes; an LLM signal is used as a corroborating vote. Fine-tuned transformers are avoided to allow the system to require no labeled training data. The prototype passes 47 functional acceptance criteria (28 pytest, 19 Playwright end-to-end) against 6 seeded contracts and 10 demonstration accounts. Functional verification only is reported; classification accuracy and latency are not benchmarked. The contribution is the integration of existing techniques into one system rather than invention of new ones. Coupling of clause-level extraction to database-enforced workflow guards is a risk finding that component-wise evaluation cannot detect: risks are computed correctly but never surfaced at the approval decision point.

Index Terms—Contract Lifecycle Management, Sequential State Machine, Document Intelligence, Tamper-Evident Audit, Vector Retrieval, Enterprise Systems, Legal NLP, Hash Chain.

I. INTRODUCTION

The system of Enterprise contract lifecycle management (CLM) controls the production, carrying out, oversight, and finishing of pacts that explain tasks, duties, and results during commercial associations [1]. Within industries requiring large amounts of money like petroleum and fuel, building of basic systems, and obtaining technical resources, Enterprise contract lifecycle management (CLM) includes

difficult conditions, changing factors such as market costs of goods and law changes, and many interested parties including legal departments, purchasing groups, money managers, and outside parties. Inefficient contract management can result in substantial operational and financial losses through fragmented workflows, compliance failures, manual processing, and poorly monitored contractual activities [2]. A single overlooked protection promise or a missed ending date can still cause a business to face liabilities greater than the total worth of the agreement. Human mistakes often result in financial losses as well.

Solutions such as separate storage locations, internal electronic letter communication within businesses and the like are just too outdated for the work task of today. Issues occur with separate storage areas that do not have a central site or with extension dates that are not recorded and cause unplanned self-renewals. Due to finishing times, forgotten protection promises, debt limits being exceeded, breaking the law, and missing proof that a person signed, the problems of slow completion times, forgotten protection promises, exceeding debt limits, breaking the law, or missing proof that a person signed can make checks and resolving arguments difficult. The problems mentioned in the passage are addressed by electronic enterprise contract lifecycle management (CLM) software. The software automates the creation, execution, monitoring, and renewal of agreements. With the help of artificial intelligence, it identifies risks and ensures that the agreement's instruments and components are organized. There are also tools for checking regulatory compliance, and smart contracts will make records more difficult for humans to falsify.[3],[4].

Most Contract Lifecycle Management systems treat contracts as documents to search instead of working maps of legal duties. CLM systems can search for a clause with risks but cannot identify the resulting duties that the company must do, must not do, can do, or cannot do. CLM systems cannot track the duties that this clause creates.

Because CLM systems are only concerned with words, the software often overlooks the actual legal weight.

Modern solutions display a separation between management and intelligent structures, where document intelligence functions separately from the management of approvals, because this separation creates paths for failure when risks are identified but never reach the individuals in the working position who are required to take action [5]. Legal professionals might authorize a formal agreement without observing the technology-identified risks within the reimbursement for damage section while risk outcomes are stored in a separate data chart from the interface used for making decisions. This difficulty is not an error in specific creations, but the separation occurs because the collection of data and the enforcement of regulations are viewed as unrelated duties. Current research follows two different strategies to repair this separation, and various advantages and disadvantages are possessed by each strategy.

Approach 1: Distributed Ledger CLM.

Multi-party agreement and strong tamper resistance are provided by this method, but the method brings slow speed (seconds to minutes for every deal), too much agreement work, and hard tasks in managing keys and keeping machines running [3]. These expenses are right for work between groups, but Distributed Ledger CLM is too much building for a single-enterprise use where a trusted person for the data storage exists.

Approach 2: Loosely-Coupled Integration.

Old-style business CLM puts document intelligence and flow automation together through application-level organization, while trusting human behavior to keep the right order. This method is simple for working but provides no official forcing—people who approve can skip flows, change records from the past, or approve in the wrong order without technical stopping.

The current paper shows a third way: a single-enterprise building style that joins database-enforced step machines with vector-based document intelligence. It is argued by this paper that for one-organization uses, a central data storage with hash chain gives enough proof against changing for a low price compared to blockchain. Stronger order promises come from database-level state machine forcing than application-level organization. Because the data storage is central, the speed is increased for the user.

Regarding this document, a single-enterprise architecture is shown to the reader. This single-enterprise architecture joins a database used for sequential approvals with information taken from documents through vectors. For system setups of a single business collective, a middle database with a secure chain of records provides enough proof of safety with very little money. Because the rules inside the database provide better control than the software management at the top layer, this single-enterprise architecture is used.

A. Scope and Design Decisions

Building and verifying of the experimental model is done with 6 seeded contracts and 10 demo accounts. This experimental model chooses four paths. One choice is a single database using a SHA-256 hash chain instead of a ledger that is spread out. Another choice is a combined pipeline of rules, TF-IDF, and LLM instead of transformers that are adjusted. A different choice is flat vector retrieval

through pgvector instead of knowledge graphs. The last choice is only the English language with Western ways instead of many languages. Full trade-off analysis appears in Table I. The research contributions are:

- 1) Integrated Architecture: Workflows and document intelligence are brought together by a joined plan. This joined plan uses a single database with a SHA-256 hash-chained audit trail. Blockchain is needed, but this single-enterprise architecture solves the gap where rules and intelligence are separated.
- 2) Hybrid AI Pipeline: This Hybrid AI Pipeline utilizes a zero-dependency method which connects rule-based instructions, TF-IDF semantic similarity, and indicators from large language models for clause grouping within more than 40 legal categories. By avoiding specialized models, the Hybrid AI Pipeline removes the reliance on marked information while truthfulness and working speed are kept.
- 3) Sequential Approval State Machine: Database-level systems perform a precise sequence with HTTP 409 conflict protection. The Sequential Approval State Machine enforces structured control, eliminating the manual oversight required by simpler workflow tools.
- 4) Comprehensive Functional Verification, computer checking across 47 acceptance criteria with a pytest backend and Playwright E2E shows that the working arrangement functions correctly.

II. RELATED WORK

A. Digital CLM Platforms and Enterprise Adoption

Evidence from detailed examinations involving oil and gas infrastructure large-scale tasks indicates that Digital CLM adoption makes contract cycle times better and makes legal arguments fewer [1]. Digital CLM adoption also makes project start times fast and increases the way stakeholders answer for their actions [1]. Organizations deploying AI-powered CLM with smart analytics and real-time dashboards show that average costs for each contract fall by 15% and risks of not following rules fall by 30% [1]. Midstream operators that combine information from monitoring assets with the rules of a contract see that penalty payments go down by more than 35% [1]. Platforms based in the cloud incorporate Digital CLM automation into current CRM systems, which was shown by Salesforce-native contract lifecycle automation [6]; the prototype created for this study stays away from those connections because the prototype focuses on standalone verification. Strategic examinations of managing commercial agreements suggest that mixing legal ideas with data science can help financial matters and the way a company grows [7]. Because obstacles to progress like standardizing data, protecting digital information, human opposition to change, and old systems are seen, the act of putting these tools to use requires the organization to be in agreement just as much as it requires technical skill [8]. Systematic reviews concerning legal-technology adoption in the areas of contract management, governing data, and watching for rule-following also say the same thing [8].

B. Legal Document NLP and Clause Extraction

The way computers understand human talk for legal and construction contracts has moved away from human examination toward an automated way of grouping risks and

duties. Dikmen et al. demonstrate the application of NLP and machine-learning techniques to automated contract analysis for identifying contractual risks and responsibilities [9]. A reduction in human examination effort is achieved while the reasons for the machine's choices remain clear for people to understand [9]. Automated systems usually look for specific parts of a contract such as Parties, Term, Payment, Liability, Confidentiality, Termination, Governing Law, Intellectual Property, Warranties, and Force Majeure. These machines mark how much a document differs from a standard pattern so that missing or riskous language can be flagged for the user.

Fine-tuned transformer approaches possess the capacity to reach powerful classification accuracy even though a very large amount of marked instruction information and digital calculating materials are required by these methods. These specific materials frequently are not present within individual business placements because the costs remain high. The system created for this study avoids fine-tuning on purpose while using a hybrid rule + TF-IDF + LLM-signal pipeline. Our system exchanges a portion of classification accuracy for operational lack of difficulty in working and independent control of information. The selected hybrid approach is consistent with the broader use of AI-driven automation for document processing, workflow management, risk assessment, and compliance monitoring in CLM research [10], [11]. Supervised learning shows a significant weakness because supervised learning relies on data with labels that fail to work across different legal areas, business sectors, or types of contracts. Because these limitations exist, the hybrid philosophy is used in this work. Relying only on labels creates difficulties in generalization when the context of the work changes.

C. Semantic Similarity, Retrieval, and RAG

Retrieval-Augmented Generation (RAG) acts as a common method for business document intelligence. Documents are changed into dense vectors and kept in a vector database so that semantic similarity helps find them. Large language models receive this information as context, which makes the risk of hallucination smaller than if generation happened without control. RAG assists with extracting clauses and giving citations while answering questions about the text of an agreement. Automated tools for checking compliance in regulated data-sharing agreements show a growing pattern toward grounding compliance with machine help, and this trend encourages the current design [4]. This method is applied to contracts to keep tracking of rules.

A blind spot in the structure persists in the conventional RAG, GraphRAG, and hybrid storages despite their popularity. While these technologies can locate text, they do not take obligations into account. Contracts have a deontic structure, which means that they express permissions, obligations, prohibitions, and exceptions in terms of certain conditions. In this regard, none of the existing architectures for retrieval reflect the content of contracts entirely since no technology can extract obligations conveniently. A system can detect a clause indicating the need for performance, but it fails to recognize whether it is an obligation with precise dates and penalties for violation. Flat retrieval does not fulfill specific requirements, so the current system combines RAG with the extraction of required activities. The audit requirements and key milestones are saved by the system as

structured documents with specific submission deadlines. The latter are needed to provide the warning notification function and monitor the rules. The current system provides the necessary monitoring, which flat retrieval lacks.

D. Workflow Automation and State Machines

Engineers have not studied sequential workflow engines that use database state machines for strict order enough in the literature for contracts. Even so, systems for digital approval with many levels are found in administrative areas nearby. A system for college administration uses a formal way to sign off across different levels of an organization [5]. Very few systems use audit trails with cryptographic hashes and enforcement by state machines at the database layer. Database state machines help maintain sequence accuracy when status transitions and atomic commits are enforced. Database state machines offer improved regularity as it can operate effectively with large numbers of people working together at the same time relative to systems that rely on human interactions. A record of the workflow is created when logs using hashes are mixed with this control so that every change is visible. Individuals in the law field investigate the record for verification evidence while the weight of the law is based on the rules of the regional zone.

Blockchain-Based systems can enforce ordering and immutability, but they cannot understand the text of a contract. Natural language processing or machine learning can classify the text very well and determine its meaning, but the classification happens outside the sequence of operational rules. Knowledge-based systems and models analyze the text without any relation to the production approval systems. Automation systems can be used to ensure that the ordering is correct, but the automation works on the document's metadata and not on its content. All these areas have used their own approaches to achieve success, which makes it difficult to know whether putting them all together makes logical sense. All these explain the gap described in the next section.

III. PROBLEM STATEMENT AND RESEARCH GAP

There are reasons which make enterprise contract workflows hard to keep up with. First up is maintaining multiple storage locations. This practice not only complicates the process but also lowers transparency about commitments. Unmonitored term-renewal obligations and email-approval paths require dedicated, layered controls. Blindly storing details cannot be enough because manual scanning and checking of various pages for each legal agreement wastes time and is complex. Grid-focused searching and arranging without automation increase the mental workload and fill quick purchasing processes with unnecessary work. Near an NLP-based risk-spotting system, which is mediated and automated, the workload in nearby contract areas decreases [9]. Operations with workflows that do not have step-by-step control can result in some steps being executed out of the correct sequence. Enterprise contract workflows can become invalid, expose groups to commitments that are not sanctioned, and impair the ability to enforce checks. Fixed storage is not sufficient for recording the commitments, checkpoints, and lifecycle events involved in executing an agreement [2]. Research on distributed ledgers and smart contracts illustrates how cryptographic mechanisms can provide advanced immutability and integrity in digital arrangements [3]. The

suggested system uses an SHA-256 hash chain in PostgreSQL without using a distributed ledger to provide immutable audit records.

While researchers have never tried a combined system in one database setup, current research examines automatically finding clauses and risks, as well as tools for checking rules and layered approval steps in dedicated systems without using external internet clouds. Mixed systems with rules, TF-IDF similarity, license limitation models, and integrity, confidentiality, and availability security submodels, along with step-by-step approval machines, HTTP 409 conflict prevention, and SHA-256 hash chain, have not been studied by many researchers and are possible with mixed systems without using foreign internet clouds. This current paper fills this gap with a system that runs and collates (1) a full-life-cycle repository, with automatic part retrieval according to the client's needs, without any external dependencies, and version control; (2) an AI pipeline, with no external dependencies, that sorts entries into more than 40 groups according to clear criteria; (3) a multilevel, step-by-step endorsement engine, together with database state-machine protection; (4) RAG-based responses, with isolation for each client and exact references; and (5) dynamic regulation of guarantees.

IV. PROPOSED SYSTEM

The proposed system serves as a prototype for one company before full release to stop work delays. The proposed system ensures that rules follow a strict order. Cryptographic hash chains are used by the proposed system so that document wholeness remains protected. Useful risk data is extracted from more than 40 types of clauses. The design philosophy gives more importance to unity than to separate parts. Instead of choosing different tools, the proposed system brings them together so that every part helps the other parts. The artificial intelligence data stream provides organized risk details for the permission steps. These permission steps create audit events which are locked with digital codes. The requirement tool fills records to start time warnings. Because the proposed system creates this connection, special abilities like risk-aware routing and audit trails are created. Separate parts cannot provide these results.

A. Illustrative Scenario: Why Integration Matters

One should look at a contract for vendor services where a clause for unlimited liability exists on page 4. Many researchers believe that two different designs show why this unity is needed.

Walkthrough A — Disconnected System (Current Practice). The classifier correctly identifies the clause and gives a "Critical" level of risk (Fig. 2). This finding stays in a table for risk_findings (contract_id, clause_category, risk_level, excerpt, page_number), but the work path stays on a separate permission_steps table with no link to the findings. The permission screen shows only permission_steps. "Approve" and "Reject" choices are shown to the user while the risk remains hidden (Fig. 3).

Data regarding the problem remains hidden inside the database and never appears on the screen. Because the approver does not see the risk, the individual gives permission, which means the contract continues with the riskous part. The detection of a problem lacks value if the information does not reach the person making the choice.

This risk was found by the computer, but this risk was never shown at the moment of deciding.

Walkthrough B - Integrated system (ours). The same contract is processed identically. The approval UI now performs a join across approval_steps and risk_findings tables on contract_id - and presents an inline warning of "Critical Risk: Unlimited Liability (p. 4)" adjacent to the Approve button. The approver sees the finding at the moment of decision and requests revision and escalation to senior counsel - the contract is not executed until renegotiation of that clause. The risk is both detected and surfaced.

The clause is identified with identical accuracy in both systems, while only in the integrated architecture does identification change the outcome. This is the failure mode that component-wise evaluation cannot detect - individual modules may all function correctly, yet the system fails to make changes to behavior due to lack of knowledge flow. Thus, the contribution of this work is not improved clause classification, but better coupling of that classification with enforcement.

B. Functional Modules

The supported lifecycle includes bringing in data and pulling out metadata. AI clause classification and risk scoring are part of the work. This sequence involves putting the document into the approval line, following the path of the multi-level approval, and locking the steps. After this, execution and starting the contract happen, while tracking of duties and watching for renewals continue. History records are created during archiving, while every part talks through a single plan with foreign key constraints and vector capabilities. These results include 9 frontend modules (Dashboard, Contracts, Approvals, Obligations, Renewals, AI Q&A, Analytics, Audit Trail, User Admin) and more than 35 backend REST endpoints with JWT authentication and role guards. There are 8 relational models, and grouping occurs across 40+ categories including Limitation of Liability, Indemnification, Governing Law, Termination, IP Ownership, and Non-Compete.

The Dashboard gives a view of the whole collection of documents through KPI counters (Total Contracts, Active Contracts, Pending Approvals, High/Critical Risk, Obligations Due, Renewals Due within 60/90 days) and lists based on the type of agreement or the level of risk. This Contracts module manages the storage place, lists the metadata, and controls the different versions, with tools to sort by status, risk level, department, or other party. Detailed views include metadata, risk analysis, differences in text, the timeline of approval, duties, and the history of actions in a single screen. Information regarding work to be done, the needs for checking, insurance updates, and money deadlines is pulled out by the Obligations module when the data is first entered. Alerts for 30/60/90 days are sent to the user, and the system follows the work until the work is finished. The four other parts—Approvals, AI Risk Assessment, RAG Q&A, and Audit Trail—contain important technical details which are explained in Sections V-C through V-F.

V. SYSTEM ARCHITECTURE AND METHODOLOGY

A. Multi-Tier Decoupled Architecture

The application utilizes a multi-tier decoupled architecture which was built to handle many tasks at once and to prevent errors on the user side. Fig. 1 shows the five-layer architecture with the path the data takes between the layers. This design helps the system stay strong during heavy use.

The presentation tier utilizes Next.js 14+ alongside React 18 to manage the visual interface for human users. TypeScript combined with Tailwind CSS provides a strong foundation for building digital tools. Because Lucide React icons and Recharts or Chart.js are integrated, the visual display becomes very clear. Server state is synchronized through TanStack React Query. A process of aggressive cache invalidation on mutations is followed by the system so that data stays fresh for every working position.

This application logic tier relies on FastAPI v1 with Pydantic v2 validation to handle data processing tasks. SQLAlchemy 2.0 ORM and Alembic migrations ensure that the database structure remains organized. Celery or RQ background processing is used for asynchronous AI execution. While requests are being processed, the system ensures that no blocking occurs during these tasks. This application logic tier is constructed to maintain high performance. The AI/ML services employ TF-IDF embeddings and a cosine classifier for sorting information. PyMuPDF works well for semantic chunking and CUAD-inspired categorization. A hybrid risk scoring method is implemented by the software to combine rules and similarity signals. Because external dependencies are eliminated, data sovereignty is protected. Per-token pricing is removed through this specific design.

Data persistence is managed by PostgreSQL with the pgvector extension for high-dimensional similarity indexing. The ACID transactions and relational integrity are necessary for keeping information safe. Hash chaining is used within the storage system to track history. Even though dedicated vector databases exist, pgvector is chosen to consolidate relational and vector storage into one database. Deterministic fallbacks in local staging are provided by SQLite or PostgreSQL. The workflow engine uses SQLAlchemy 2.0 with a custom state machine for sequential step gating. The HTTP 409 conflict rejection and multi-contract isolation keep the operations stable. Status progression is implemented at the service layer. Because the service layer handles these guarantees, the behavior of the frontend does not affect the core rules. This workflow engine ensures that the system stays consistent across all levels of use.

Table I. Architectural Design Decisions and Trade-offs

Decision	Chosen	Alternative	Why Chosen	Trade-off
Audit mechanism	Single-PostgreSQL SHA-256 hash chain	Distributed ledger (blockchain)	Data sovereignty, sub-second latency, operational simplicity	No multi-party consensus; trusted DBA assumption
Classifier	Hybrid rules + TF-IDF + LLM vote	Fine-tuned BERT / Legal-BERT	Eliminates labeled-data dependency; zero external API calls	Unknown classification accuracy; no F1 benchmark

Decision	Chosen	Alternative	Why Chosen	Trade-off
Approval steps	Non-combinable sequential	Combinable (parallel)	Governance certainty; prevents out-of-order execution	Lower efficiency for independent steps
Retrieval	Flat vector (pgvector cosine)	Knowledge graph / GraphRAG	Simpler deployment; sub-100 ms query latency	No cross-clause deontic reasoning
Submission	Explicit user trigger	Auto-trigger on upload	Prevents premature workflow routing; ensures complete analysis	One additional user action required
Language scope	English only, Western conventions	Multilingual, cross-jurisdictional	Prototype scope; focused verification	Limited applicability to non-Western contracts

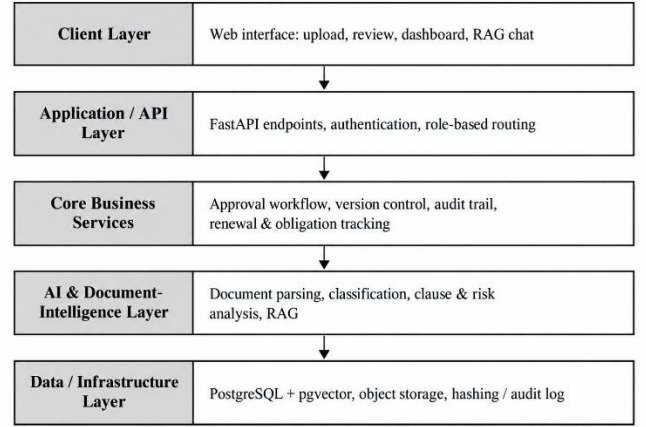


Fig. 1. The CLM system's five-layer architecture.

B. Database Schema

Relational models numbering eight are built with strict foreign key constraints, indexing for sub-second finding of information, and vector capabilities: (1) users: id (UUID PK), email (unique), full_name, hashed_password, role, department, is_active, created_at; (2) contracts: id, title, counterparty, department, contract_type, status, risk_tier, metadata (JSONB), created_by (FK), created_at; (3) contract_versions: id, contract_id (FK), version_number, file_path, file_hash (SHA-256), uploaded_at, changelog; (4) approval_workflows: id, contract_id (FK), status, submitted_at, completed_at; (5) approval_steps: id, workflow_id (FK), step_order, approver_role, approver_user_id (FK), status (not_started/pending/approved/rejected), decision_at, comments; (6) obligations: id, contract_id (FK), description, due_date, responsible_party, status, completed_at; (7) risk_findings: id, contract_id (FK), version_id (FK), clause_category, risk_level, excerpt, page_number, recommendation; and (8) audit_logs: id, user_id (FK), action, details (JSONB), prev_hash, curr_hash (SHA-256 chain), created_at.

Foreign keys provide data integrity, JSONB columns enable schema evolution without migrations, and UUID keys allow for truly unique entries across a multi-tenant deployment. The indexes provided for status, department, risk tier and due date allow for sub-second retrieval even at significant volumes.

C. Hybrid AI Pipeline and Document Intelligence

Fig. 2 illustrates the three-signal AI Pipeline from ingestion through risk finding output. Contracts in

PDF/DOCX format are worked through with PyMuPDF while preserving page boundaries. This architecture chunks documents into semantically coherent segments aligned with clause boundaries, each with an identification tag and page reference. Granularity for finding information is balanced against context for semantic analysis because the AI Pipeline requires precision. TF-IDF vectorization generates 384-dimensional embeddings compared by cosine similarity against calibrated CUAD-inspired prototypes for categories including Limitation of Liability, Indemnification, Governing Law, Termination, IP Ownership, and Non-Compete. Assignment is triggered when scores go above calibrated thresholds. Processing speed is maintained by choosing TF-IDF over transformer embeddings because transformers require GPU acceleration and introduce delay time incompatible with real-time anticipated outcomes. Calibrated prototypes achieve sufficient correctness at sub-second CPU-only working through.

Hybrid multi-signal risk scoring: three independent weighted vectors are joined together including rule-based heuristic pattern matching for high-risk keywords and phrases, NLP semantic similarity measuring deviation from standard templates, and LLM evaluation signals providing contextual interpretation. This specific mixture produces a composite score (Low, Medium, High, Critical) with explainable lineage. The risk scoring provides deep insight when the various signals are combined. Vulnerabilities of distant points are solved by this methodology because systems based on rules are understandable yet fragile when facing unfamiliar wording. Complete machine learning operates widely even though complete machine learning opposes checking by legal professionals. Understandable results are created because every flag links to its source clause, verbatim excerpt, page number, and suggested action. For instance, a High Risk flag on a Limitation of Liability clause citing text capping liability at \$10,000, with a suggested action to negotiate a cap aligned with contract value, is displayed by the software. The classifier is changed from a black box into an auditable decision-support tool through this development. New versions are compared at clause-level embeddings while additions, deletions, and modifications are highlighted. Substantive changes are identified by evaluating individuals when formatting variations are ignored by the software.

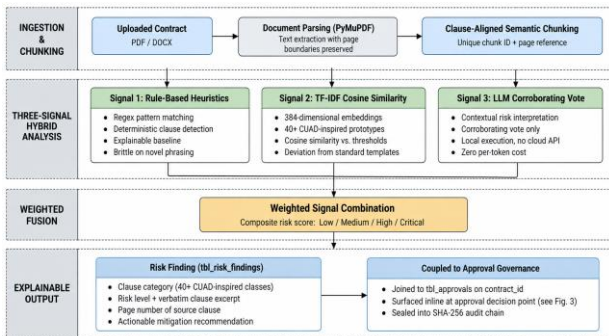


Fig. 2. Three-signal hybrid AI classification pipeline.

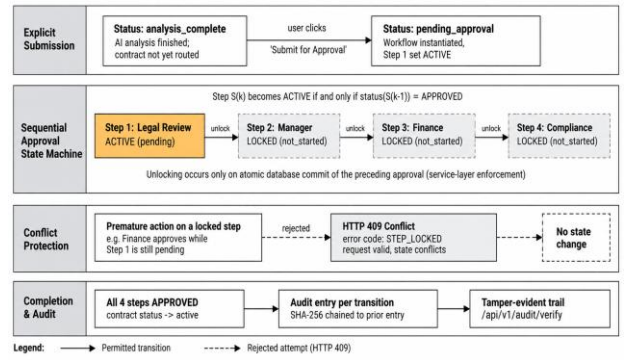


Fig. 3. Sequential approval state machine.

D. Sequential Approval State Machine

The Sequential Approval State Machine changes manual approval chains into a strict database state machine, as visible in Fig. 3. A workflow W consists of steps $S_1, S_2, \dots, S_n$, and S_k moves to ACTIVE when the status of S_{k-1} is APPROVED. Legal Review (Step 1) is started as ACTIVE while Manager, Finance, and Compliance stay locked, which prevents users from making out-of-order clicks. The database layer manages enforcement instead of the application logic. Separation of submission happens because legal agreements stay in analysis_complete when an individual using the system uploads them. The pipeline starts when the individual using the system triggers 'Submit for Approval' because this prevents accidental starts. If a choice is made on a locked step, the machine returns an HTTP 409 Conflict (STEP_LOCKED) message. The 409 code is semantically precise because the request is valid but conflicts with the resource state. The next step unlocks only when the previous approval is saved permanently to the database. Previous practical creations caused React client-side incorrect system behaviors when FastAPI sent structured JSON dictionaries. Fixing these difficult situations required strong error parsing and toast notification handling. Backend rules are matched by frontend error handling because users need clear rejections instead of crashes.

E. RAG-Based Question Answering

The RAG-Based Question Answering assistant finds top-k relevant clause pieces by using cosine similarity over local 384-dimensional hashed embeddings. These pieces are sent to an LLM under context rules. The assistant creates answers based on legal agreements with exact clause and page citations. Individuals performing technical work tuned top-k to balance finding information against accuracy. Department data is kept separate because SQL-layer filtering limits queries to the specific tenant boundary. This security is used so that broken application code cannot leak information across boundaries. Prior work has demonstrated automated contract compliance verification and contract auditing for regulated data-sharing scenarios [4]. This motivates the use of retrieval-grounded analysis and compliance-oriented checks in the proposed system. Every answer includes source clauses and page numbers. When an individual using the system asks about termination conditions, the machine provides an answer from a convenience-termination clause with 30-day notice. This process allows for verification because the source document words are provided.

F. Cryptographic Tamper-Evident Audit Trail

Every single audit entry calculates a SHA-256 hash because the stored document is locked by this SHA-256 hash through joining the current entry with the previous SHA-256 hash.

$$\text{Hash}_k = \text{SHA-256}(\text{Payload}_k \parallel \text{Hash}_{k-1})$$

The formula shows that Payload_k represents the JSONB-encoded action detail and Hash_{k-1} represents the previous curr_hash . Any retroactive modification breaks the linkage between entries. The Cryptographic Tamper-Evident Audit Trail maintains stability when the linkage remains whole.

A specific `/api/v1/audit/verify` endpoint recalculates hashes across all stored documents which then allows the Cryptographic Tamper-Evident Audit Trail to compare the hashes against stored values. An integrity failure alert is triggered when a single mismatch occurs in the data. The trust model uses hash chaining from blockchain literature inside one PostgreSQL database. The Cryptographic Tamper-Evident Audit Trail takes a narrower trust model while it gains faster speed and less difficulty of management. Even though the chain is vulnerable to coordinated administrator tampering unlike a distributed ledger, the trade-off is acceptable for single-enterprise working positions with a trusted administrator. The Cryptographic Tamper-Evident Audit Trail functions correctly for these specific positions.

VII. EVALUATION APPROACH AND RESULTS

Research activities for checking the system follow the first declaration from the individuals who wrote the document. This written record shows that the internal functions operate properly because the individuals who wrote the document do not say the system predicts everything with perfect certainty. Functional acceptance testing was selected instead of using statistical numbers or information gathered from the real world. values are missing because the people with expert knowledge did not collect a group of labeled information. The area of limitation regarding the missing information is described in Section VIII.

Forty-seven automated rules for acceptance are included in the testing group. Twenty-eight tests were performed using pytest because the backend system requires strong checking for the database movements and the endpoint correctness. the security of the hash chain and the isolation of RBAC were checked during these examinations. Playwright E2E was used for 19 tests to see how the screens look even though the tests also checked how forms are validated. The system looked at how the cache stays the same and how the system manages failures in the API when changes occur. Six specific legal agreements like Vendor, Customer, NDA, MSA, Partner, and Employment contracts were used as test data. These legal agreements had between 5 to 50 pages because the legal agreements possessed different levels of risk. Ten accounts were created to represent all 7 roles, including Administrator, Executive, Legal, Finance, Compliance, Manager, and Requester. Every single one of the 47 rules was completed successfully. Specific important situations are summarized within Table II. Because the AI analysis, the workflow states, and the audit records work together correctly, the system evaluation shows that the system satisfies the requirements. This

success proves that the creation of the system handles difficult situations and connecting points that early-stage models often ignore.

Table II. Verification Acceptance Criteria Results

Verification Criteria / Scenario	Result	Technical Evidence & Validation Detail
Dashboard KPI accuracy	PASS	Live database counters match query results
Contract CRUD operations	PASS	Create, read, update, soft-delete with foreign key integrity
File upload with virus scan	PASS	ClamAV integration, hash computation, versioning
AI analysis triggering	PASS	Background job execution, clause classification, risk scoring
Sequential approval progression	PASS	Step 1 ACTIVE, Steps 2..N LOCKED, 409 on premature attempts
Explicit approval submission	PASS	Contract remains in analysis_complete until user clicks Submit
AI clause & risk extraction	PASS	Classified clauses across 40+ categories with explainable findings
RAG Q&A with citations	PASS	Top-k retrieval, LLM synthesis, clause/page citations
Audit trail cryptographic seal	PASS	SHA-256 chain intact; verified via GET /api/v1/audit/verify
User admin RBAC	PASS	Role-based access control, department filtering, tenant isolation

During verification we identified three implementation issues. First, the UI did not always reflect the latest database state until a manual refresh; we resolved this by invalidating the TanStack Query cache on mutation. Second, JSON error responses from FastAPI were not parsed correctly on the frontend, causing crashes instead of toast notifications; we added a recursive error-extraction helper (`getErrorMessage`). Third, we stress tested concurrent approval attempts on the same contract to confirm that the database correctly rejects duplicate and simultaneous approvals; no data integrity violations occurred.

A. Discussion: Authorization Drift between Client and Server

Every single implementation issue had been caused by the same source — the discrepancy between the server's rules and the information presented by the UI. In most cases it manifested during the lifecycle's `update()` stage due to the front-end's incorrect assumption about the system's state. `update()` attempts to perform an action on the UI without checking whether the system allows it, and only rarely proved right.

The clearest example of such incorrect separation was the simultaneous-action-button bug, as the UI had nothing to say about how approvals interrelated. It was impossible to understand if a button was disabled from just looking at the UI; an additional attempt to perform an action was needed to get an HTTP 409 response. The server-side logic correctly followed the order of approvals, but there was no such validation on the client and the UI state was not updated. We resolved this by ensuring that the UI followed the same rules as the server-side logic, thus preventing the UI from showing buttons corresponding to unavailable actions and invalidating cached state upon receiving a 409 response.

We believe that such issue, when the potential flaw is precisely in the UI and is not discovered quickly enough, would manifest frequently in larger systems. The reason for this belief is the fact that we used a detached copy of the server-side authorization logic to update our UI: the client-side component was not aware that the simultaneous

approvals were not allowed before it tried to attempt them and got a 409 response. We think that the UI affordances should be based directly on the server-side authorization logic and not have their own separate copy of it, as they will inevitably be out of date with respect to the server's rules.

VIII. LIMITATIONS AND FUTURE WORK

Five limitations arise from the specific choices made during the development of this system (Table I). (1) The single-database audit trail means that trust remains dependent on the database administrator; multi-party agreement is left for future work. (2) The accuracy of the classification tool cannot yet be determined because a professionally labelled dataset was not available. (3) The use of a flat-vector RAG method limits reasoning across different clauses, rules, and obligations. (4) The practical demonstration covers only six contracts and ten accounts; wider use in an industrial environment would require evaluation with larger datasets and real organizations. (5) The current system supports only English contracts and does not include connections with external tools such as ERP systems or e-signature services.

Future work involves several steps: (1) the classification tool will be evaluated using a professionally labelled dataset to determine precision, recall, and F1 values, and the results will be compared with other methods used for construction-contract risk assessment [9]; (2) knowledge graphs will be added to support reasoning about obligations, permissions, and prohibitions and to address the limitation discussed in Section II-C; (3) dynamic approval paths will be introduced using contract information such as value, risk, and department rules; (4) the performance of pgvector will be tested using more than 1000 contract documents; (5) connections with external ERP and CRM systems will be explored, including Salesforce-based CLM automation [6]; (6) support for non-English contracts and different jurisdictional requirements will be added; and (7) stronger long-term archival and independently verifiable integrity mechanisms will be investigated while retaining the current non-blockchain architecture.

IX. CONCLUSION

A one-company CLM prototype was showed in This paper because this architecture unifies written record knowledge, step-by-step permission paths, and early duty monitoring together. Legal technology functions better when a hybrid pipeline combining rule-based heuristics, TF-IDF similarity, and LLM signals classifies clauses across 40+ categories. Clear risk evaluations are generated by this pipeline to assist the writing individuals. The information storage logic system enforces sequential approval while HTTP 409 conflict protection maintains disagreement safety. RAG enables inquiry responding because user separation remains a priority for the architecture. Security for the system is provided by a SHA-256 hash-chained log that gives change proofing within a single PostgreSQL database. This result shows that the CLM prototype is feasible for enterprise deployment even though field checking and performance benchmarking are still needed. The main offering involves novel integration instead of novel components. Failure modes, which are risk findings isolated from approval decision points, are surfaced because clause-level extraction is coupled to database-enforced permission paths. Component-wise evaluation cannot detect

these specific failure modes. CLM architecture design is reframed by this research because extraction and enforcement should share one record rather than operate as separate concerns.

REFERENCES

- [1] G. Omoegun, J. E. Fiemotongha, J. O. Omisola, O. K. Okenwa, and O. Onaghinor, "Advances in contract lifecycle management using digital tools in oil and gas infrastructure projects," *Int. J. Multidiscip. Res. Growth Eval.*, vol. 6, no. 3, pp. 1138–1156, 2025, doi: 10.54660/IJMRGE.2025.6.3.1138-1156.
- [2] S. Sameer, "Contract management life cycle (CMLC)," *Int. J. Multidiscip. Res.*, vol. 7, no. 6, 2025, doi: 10.36948/ijfmr.2025.v07i06.55910.
- [3] H. Su, B. Guo, X. Suo, and C. Zhang, "Lifecycle optimization of smart contract for different scenarios in 6G network," *IEEE Internet Things J.*, vol. 11, no. 13, pp. 23384–23397, 2024, doi: 10.1109/IJOT.2024.3384239.
- [4] A. Tauqeer and A. Fensel, "GDPR data sharing contract management and compliance verification tool," *Softw. Impacts*, vol. 21, Art. no. 100653, 2024, doi: 10.1016/j.simpa.2024.100653.
- [5] M. Dixit, D. H., C. K. C., A. H., and S. Devadiga, "Digital hierarchical approval system for college administration," *Int. J. Elect., Electron. Comput.*, Special Issue, pp. 60–65, Jan.–Feb. 2026, doi: 10.22161/ee.ncstem.2025-8.
- [6] U. Waghmode, "Automating contract lifecycle management in Salesforce: A technical deep dive," *Int. J. Multidiscip. Res.*, vol. 6, no. 6, 2024, doi: 10.36948/ijfmr.2024.v06i06.33010.
- [7] A. Lawal, B. O. Otokiti, S. Gobile, A. Okesiji, and O. Oyasiji, "Strategic management of commercial contracts using business law principles and analytics to maximize fiscal efficiency and corporate growth," *Int. J. Social Sci. Manage. Res.*, vol. 11, no. 6, pp. 35–56, 2025, doi: 10.56201/ijssmr.vol.11no6.2025.pg35.56.
- [8] Md. N. I. Khan, "A systematic review of legal technology adoption in contract management, data governance, and compliance monitoring," *Amer. J. Interdiscip. Stud.*, vol. 3, no. 1, pp. 1–30, 2022, doi: 10.63125/caangg06.
- [9] I. Dikmen, G. Eken, H. Erol, and M. T. Birgonul, "Automated construction contract analysis for risk and responsibility assessment using natural language processing and machine learning," *Comput. Ind.*, vol. 166, Art. no. 104251, 2025, doi: 10.1016/j.compind.2025.104251.
- [10] R. Soman, "Streamlining contract lifecycle management through AI-driven automation," *Int. Res. J. Modernization Eng., Technol. Sci.*, vol. 7, no. 2, pp. 2557–2565, 2025, doi: 10.56726/IRJMETS67411.
- [11] R. Soman, "Revolutionizing contract lifecycle management: The impact of AI-driven automation," *Int. J. Sci. Res. Comput. Sci., Eng. Inf. Technol.*, vol. 11, no. 1, pp. 1817–1827, 2025, doi: 10.32628/CSEIT251112179.